CST2550 - Coursework 2

Design, development, testing, and management of software solution production

Group Members		
Student ID	**Student Name/Surname**	**Role**
M01015301	Nunkoo Namrata-Riya	Scrum Master
M01038115	Hau Yam Seung Mélanie	Secretary
M01014266	Coret Mégane	Developer 1
M01026956	Ramtohul Syrielle	Developer 2
M01015305	Sedoyeka Rayan	Tester

Submission Date: 15/04/26
Lab Tutor: Dr. Gina Soopramanien

Contents

1. Introduction 4
1.1 Project description 4
1.2 Project Scope 4
1.3 Users 4
1.4 Report Layout 4
2. Design 4
2.1 Data Structures 4
2.1.1 Array 4
2.1.2 Dictionary <MyDictionary> 5
2.1.3 List <MyList> 5
2.1.4 Linked List <MyLinkedList> 6
2.2 Pseudocode 6
2.2.1 List 6
2.2.2 Linked list 7
2.2.3 Dictionary 7
2.3 Database Design 8
3. Testing 8
3.1 Test Case 9
4. AI agent 9
4.1 Design 9
4.2 Functionality Overview 10
4.3 Implementation Overview 10
5. Conclusion 10
5.1 Summary of Work Done 10
5.2 Limitations and Reflection 10
5.3 Future Improvements 10
5.4 Change in Approach 10
6. References 11

Table of Figures

Figure 1: Entity-Relationship Diagram 8

Table of Tables

Table 1: Extract of Time Complexity Analysis for List 7
Table 2: Extract of Time Complexity Analysis for Linked List 7
Table 3: Extract of Time Complexity Analysis for Dictionary 8
Table 4: Test Cases for the Core Functionalities of the Betting System 9

1. Introduction

1.1 Project description

The Football Betting System application is designed to simulate a football betting platform that allows users to register and log in, view upcoming football matches, place pre-match cumulative bets and monitor the outcomes of their bets. Users can also edit their profiles, and deposit or withdraw money from their digital wallets. The system incorporates authentication mechanisms, automated bet validation and payout calculations. In addition, an administrative interface is provided to manage users and matches, as well as to access financial reports.

1.2 Project Scope

The scope of this project includes the design and development of a prototype for a Football Betting System with key functionalities such as user authentication, match browsing, bet placement, viewing bet history, and digital wallet management. It also includes the implementation of custom data structures, algorithms for odds calculation, a match simulation system and a data analyst AI agent, supported by a structured database for storing information. However, regulatory compliance and real-money deployment are outside the scope of this implementation

1.3 Users

The primary users of the Football Betting System are individuals who register on the platform to place bets on football matches and track their winnings, along with the administrator who is responsible for managing matches, users and financial data.

1.4 Report Layout

This report presents the system design, including data structures implemented and their justification, database design, testing methods and test cases, followed by an overview of the AI Agent implemented. It concludes with a summary of the work completed, limitations of the system and proposed future improvements and change in approach.

2. Design

2.1 Data Structures

2.1.1 Array

Used for: Storing leagues.

Why it has been chosen:

The number of leagues is small, fixed and known in advance. Moreover, leagues are defined only once with no deletion or update operations. Hence, a simple array is sufficient for storage.

Performance benefit:

Arrays allow simple traversal when displaying leagues and since there are only a few, the traversal time O(n) is negligible.

2.1.2 Dictionary <MyDictionary>

Used for: Teams grouped by Team ID, Matches grouped by League ID, Game Results grouped by Game id, Odds grouped by Game ID, Players grouped by Team ID and Teams grouped by League ID

Why it has been chosen:

It provides fast key-based access to related data and allows quick retrieval using unique identifiers as keys (GeeksForGeeks, 2026). By using keys such as League ID or Game ID, the system avoids the need for filtering or searching through entire collections each time data is requested. For example, grouping matches by League ID allows for faster retrieval of games in a selected league.

Compared to lists or arrays, which require O(n) linear search time to locate specific items, dictionaries reduce retrieval time by enabling direct access to values through keys. These values can also contain collections such as lists, allowing efficient grouping of related data. They also improve data organization by logically grouping related entities making the system easier to manage.

Performance benefit:

Dictionaries provide an average time complexity of O(1) for insertion and lookup due to their key-based mechanism (GeeksForGeeks, 2026). This makes the system more efficient when retrieving values instead of traversing lists or arrays.

2.1.3 List <MyList>

Used for: Storing matches, bet slips in history, bets within historical bet slips and users

Why it has been chosen:

A list is used because the number of elements is dynamic and can change frequently. Unlike arrays, lists are resizable and allow easy addition and removal of elements, making them more suitable for this purpose. Moreover, compared to dictionaries, lists are more appropriate when key-based access is not required, and data needs to be stored in sequence.

For data displayed in chronological order, items are retrieved from the database in the required order. Where a different ordering is required, a reverse operation is used instead of full sorting as it is more efficient in this scenario.

Performance benefit:

Lists provide efficient dynamic resizing and allow fast appending of elements (O(1) time complexity). They also support efficient iteration for displaying data such as matches and bet history. Operations such as searching have O(n) time complexity, however, this is acceptable as the system filters results across the entire collection to display all matching results. Lists also support simple manipulation operations such as sorting, reversing, and filtering.

2.1.4 Linked List <MyLinkedList>

Used for: Storing bets in the current unpaid bet slip.

Why it has been chosen:

A linked list is used because bets are frequently added and removed from the slip before confirmation. Unlike arrays or regular lists, linked lists do not require shifting elements when performing deletion, making them more suitable (GeeksForGeeks, 2025).

Performance benefit:

Insertion at the head of a linked list has a time complexity of O(1) and deletion can also be performed in O(1) time since a direct reference to the node is stored for each bet suitable (GeeksForGeeks, 2025; GeeksForGeeks, 2024). Moreover, as new bets are inserted at the head, the most recently added ones appear first when displayed, eliminating the need for sorting and improving overall efficiency.

2.2 Pseudocode

The pseudocodes below are extracts of the full pseudocode and analysis which can be found on the repository in the *Project Report* folder.

2.2.1 List

```
//manually increasing the size of the array to accommodate more items
PROCEDURE Resize()
        Capacity ← capacity * 2
        CREATE  ARRAY resizedArr OF SIZE capacity
        FOR  index ← 0 to count
                resizedArr [index] <--items[index]
        NEXT
        Items ← resizedArr
END PROCEDURE
```

Process	Best case	Worst case	Reason
GetItem(index)	O(1)	O(1)	Constant time due to direct array index access.
Resize()	O(n)	O(n)	All elements must be copied into a new array, which iterates *n* times

Table 1: Extract of Time Complexity Analysis for List

2.2.2 Linked list

```
FUNCTION Remove(node) RETURNS BOOLEAN
    IF count = 0 OR node = NULL THEN
      RETURN FALSE
    END IF
    IF node.previous != NULL THEN
      node.previous.next ← node.next
    ELSE
      head ← node.next
    END IF
    IF node.next != NULL THEN
      node.next.previous ← node.previous
    END IF
    count ← count - 1
    RETURN TRUE
END FUNCTION
```

Process	Best case	Worst case	Reason
Remove(node)	O(1)	O(1)	It only performs comparisons and reassigns values of the previous and next node, without any search or traversal
Node()	O(1)	O(1)	It only assigns values directly to variables
Traverse()	O(n)	O(n)	Performs *n* times iterations as each node in the linked list is visited

Table 2: Extract of Time Complexity Analysis for Linked List

2.2.3 Dictionary

```
FUNCTION GetBucketIndex(key) RETURNS INTEGER
    DECLARE hashCode: INTEGER
    hashCode ← HASH(key)
    RETURN ABS(hashCode MOD capacity)
END FUNCTION
```

Process	Best case	Worst case	Reason
Resize()	O(n)	O(n)	All n existing nodes are rehashed into a new array that has twice the capacity of the previous one, which is a linear operation.
Traverse()	O(1) or O(n+k)	O(1) or O(n+k)	Iterates over all k slots (including empty ones) and all n stored nodes, where k is the current capacity.

Table 3: Extract of Time Complexity Analysis for Dictionary

2.3 Database Design

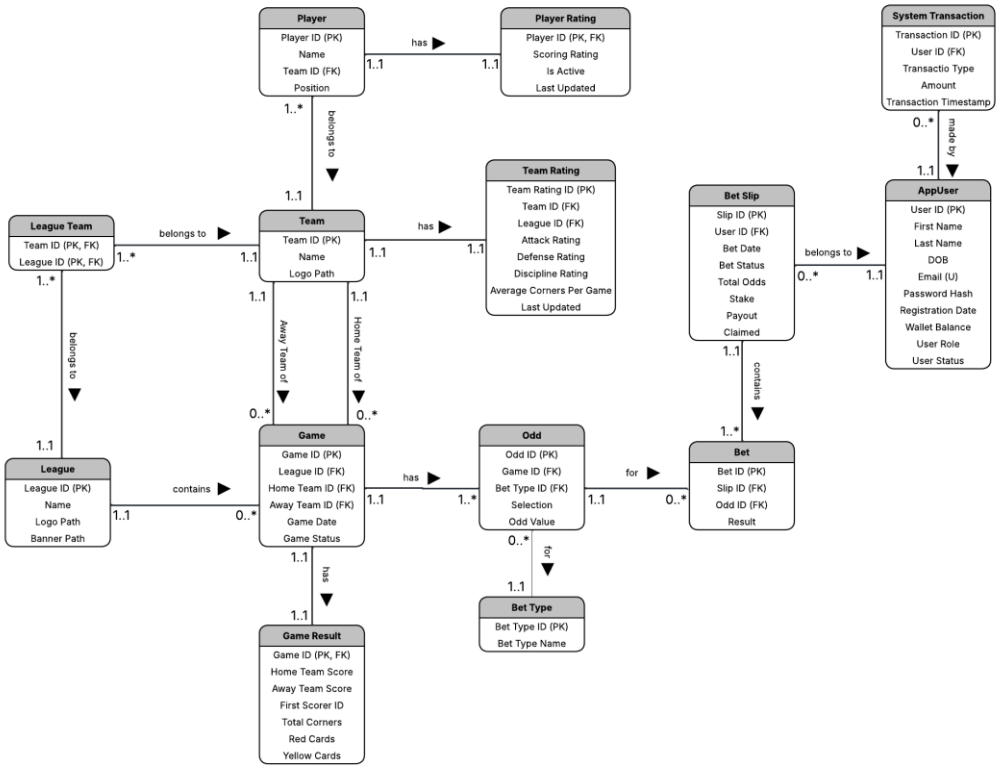

Figure 1: Entity-Relationship Diagram

Figure 1 shows the conceptual model of the database, with entities and relationships reflecting normalisation: no repeating groups of attributes (1NF), partial key dependencies (2NF) or transitive dependencies (3NF), reducing redundancy and avoiding update anomalies (GeeksForGeeks, 2026). In a real-world scenario, a Game could exist independently until a GameResult is recorded, however, in this prototype, all games have results (from CSV import or generated upon admin game insertion), hence **Game** and **GameResult** entities have a compulsory relationship.

3. Testing

The testing strategy adopted consisted of two approaches within the MSTest framework. A risk-based testing approach was used to prioritise features with the highest potential impact in the event of failure, including authentication, wallet transactions, bet placement, odds generation, and data persistence integrity. The test suite combines unit tests (validation and business rules), service-level tests (application logic and state transitions), and database integration tests (transaction and storage correctness). Due to platform limitation during runtime execution, emphasis was placed on deterministic logic checks, boundary conditions, and failure-path behaviour.

3.1 Test Case

Test Name	Number of Tests	Purpose	Outcome (Pass/Fail)
LoginAsync_SuspendedAccount_ReturnsStatusMessage (DatabaseManagerTests.cs)	1	Confirms suspended users cannot log in, enforcing account access policy.	Pass
RegisterAsync_DuplicateEmail_ReturnsDuplicateMessage (DatabaseManagerTests.cs)	1	Prevents duplicate account creation and protects identity/data integrity.	Pass
SaveBetSlipAsync_MultipleBets_PersistsAllBetRowsAndTotals (DatabaseManagerTests.cs)	1	Validates the core betting flow and payout-related persistence accuracy.	Pass
SaveBetSlipAsync_EmptySlip_ReturnsFailureMessage (DatabaseManagerTests.cs)	1	Blocks invalid bet submissions before they reach persistence.	Pass
WithdrawalOrPlaceBetAsync_InsufficientBalance_ReturnsFalseAndMessage (WalletServiceTests.cs)	1	Enforces wallet safety by preventing overdraft-like transactions.	Pass
DepositOrPayoutAsync_PayoutWithoutSlipId_ReturnsFalseAndMessage (WalletServiceTests.cs)	1	Ensures payouts require traceable slip linkage for financial control.	Pass
GetOrCreateCorrectScoreOddAsync_WhenOddIsMissing_CreatesAndReturnsOdd (DatabaseManagerTests.cs)	1	Keeps betting markets available by creating missing required odds.	Pass
AddNewMatchAsync_ValidMatchAndResult_PersistsAndReturnsTrue (DatabaseManagerTests.cs)	1	Validates admin match creation pipeline and generated odds persistence.	Pass
CheckPasswordValidity group (ValidationTests.cs)	5	Enforces password strength policy and protects user account security.	Pass
Resize_AfterManyInsertions_AllValuesRemainAccessible (MyDictionaryTests.cs)	1	Protects system stability by validating custom data structure reliability under growth.	Pass
GenerateFinancialReport ValidApiKey ReturnStructuredExecutiveSummary (AdminFinancialPage.cs)	1	Validates the AI report feature generates a usable executive financial summary from live system metrics, proving the analytcs-agen workflow works end to end.	Pass
InsertMatch_InsertsByGameDateDescending (MyListsTests.cs)	1	Ensure match ordering stay correct by date in the custom list, preventing display/processing errors in time-sensitive betting flows.	Pass

Table 44: Test Cases for the Core Functionalities of the Betting System

The test class files are in a separate project called *BettingSystemsTests.Tests*, which also uses .NET 10.0, and is part of the same solution as the *BettingSystem* project containing the main application code.

4. AI agent

4.1 Design

A data analyst AI agent was integrated using the Google Gemini API to generate contextual financial reports for administrative use, drawing directly from transaction and betting data stored in the database. The agent is invoked on demand from the Admin Financial page and produces a structured report without any manual data interpretation from the administrator.

4.2 Functionality Overview

The agent is supplied with aggregated data from two database tables: *SystemTransaction*, which records all deposits, withdrawals, bet payments, and payouts, and *BetSlip*, which holds the outcome and stake details of each user's betting activity. From this data, the agent produces a report covering four areas: a platform performance overview, a revenue and profitability analysis, a user activity and engagement summary, and actionable recommendations for the administrator.

4.3 Implementation Overview

The implementation is handled by the *GenerateFinancialReport* method in *AdminFinancialManager.cs*. On invocation, the system reads the API key from *App.Config* and queries the database to retrieve the relevant financial data. A structured prompt is then constructed by embedding the retrieved data so the agent has full context. The prompt is sent to the Gemini API, which processes the input and returns a formatted financial summary. The response is then extracted and displayed directly on the Admin Financial interface. Error handling accounts for API unavailability, such as high-traffic periods, in which case the administrator is advised to retry after a short interval.

5. Conclusion

5.1 Summary of Work Done

The system was successfully developed as a WinForms application simulating a football betting platform. It enables users to register, log in, browse matches, place various types of pre-match football bets, manage financial transactions, and track betting history. A match simulation feature, and an admin interface for managing users and matches, and viewing financial data were implemented, alongside an AI Agent for generating financial reports.

5.2 Limitations and Reflection

The system uses static odds instead of dynamic real-world data, reducing realism. Financial transactions are also simulated, with no real payment processing occurring. Moreover, user age verification relies on manual input rather than official identity validation methods, which limits accuracy and could raise compliance concerns. In addition, match results for new matches are randomly generated, and the simulation timer only runs while the application is running, reducing system accuracy.

5.3 Future Improvements

Future improvements include integrating real-world data or APIs for odds and match results, implementing secure payment gateways, and improving age verification through official NID systems. Furthermore, the simulation timer could be replaced with a scheduled background task to allow continuous database updates. Multi-user support could be implemented through techniques such as threading and database locking, allowing multiple users to interact with the system simultaneously.

5.4 Change in Approach

If this project were developed again, a more structured approach would be adopted, with better planning and system design prior to implementation, ensuring clearer definition of system requirements and better overall understanding of the system. Moreover, tasks would be allocated more effectively to allow greater focus on individual components, improving the quality and efficiency of the prototype.

6. References

Davis, M. (2021) *Building a Linked List System From Scratch in C#, Part 1.* Available at: https://medium.com/geekculture/building-a-linked-list-system-from-scratch-in-c-part-1-51aa6c68ea19 (Accessed: 26 March 2026).

Farkash, O. (2022) *How to grab an image from url asynchronously in WinForms properly using Tasks*. Available at: https://stackoverflow.com/a/70628534 (Accessed: 12 March 2026).

GeeksForGeeks (2024) *Given only a pointer/reference to a node to be deleted in a singly linked list, how do you delete it?* Available at: https://www.geeksforgeeks.org/dsa/given-only-a-pointer-to-a-node-to-be-deleted-in-a-singly-linked-list-how-do-you-delete-it/ (Accessed: 20 March 2026).

GeeksForGeeks (2025) *Advantages and Disadvantages of Linked List*. Available at: https://www.geeksforgeeks.org/dsa/advantages-and-disadvantages-of-linked-list/ (Accessed: 20 March 2026).

GeeksForGeeks (2025) *Insert a Node at Front of a Linked List*. Available at: https://www.geeksforgeeks.org/dsa/insert-a-node-at-front-beginning-of-a-linked-list/ (Accessed: 20 March 2026).

GeeksForGeeks (2026) *Dictionary in C#*. Available at: https://www.geeksforgeeks.org/c-sharp/dictionary-in-c-sharp/ (Accessed: 20 March 2026).

GeeksForGeeks (2026) *Hashing in Data Structure*. Available at: https://www.geeksforgeeks.org/dsa/hashing-data-structure/ (Accessed: 20 March 2026).

GeeksForGeeks (2026) Introduction to Database Normalization Available at: https://www.geeksforgeeks.org/dbms/introduction-of-database-normalization/ (Accessed: 8 April 2026).

Gollhardt, C. (2019) *How to hash a password*. Available at: https://stackoverflow.com/a/32191537 (Accessed: 7 February 2026).

Kanavos, P. (2022) *How to use PeriodicTimer inside of constructor?*. Available at: https://stackoverflow.com/questions/70688080/how-to-use-periodictimer-inside-of-constructor (Accessed: 22 March 2026).

Microsoft. (2026) *Indexers*. Available at: https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/indexers/ (Accessed: 26 March 2026).

Nemes, A. (2020) *Roll your own custom list with C# .NET*. Available at: https://dotnetcodr.com/2020/10/13/roll-your-own-custom-list-with-c-net/ (Accessed: 25 March).

StackOverflow Ciamej(2022) *How can I calculate the time complexity of the following pseudocode*. Available at: https://stackoverflow.com/questions/70386509/how-can-i-calculate-the-time-complexity-of-the-following-pseudocode#:~:text=1%20Answer&text=Pretty%20obviously%20the%20complexity%20is%20O(n)%20.,you%20get%20O(n)%20 (Accessed: 10 April 2026).

StackOverflow Vadim Kalinsky(2015) *what is the difference between O(nk) and O(n+k) in time complexity?* Available at: https://stackoverflow.com/questions/27301287/what-is-the-difference-between-onk-and-onk-in-time-complexity (Accessed: 13 April 2026).